Zephyr Project

IT-Schulungen.com
Technology & Transformation

Praxis: Konnektivitätsgesteuerte Anwendung

4.10

Drei Verantwortungen – klar getrennt

01	`L4-Event-Callback`	früh registriert; filtert Connected und Disconnected und stößt nur Folgearbeit an
02	`Connectivity State`	threadsicher, idempotent und unabhängig von einzelnen Event-Wiederholungen
03	`Pipeline / Transport`	Sensorik läuft; Transport folgt dem L4-Zustand

Die Fachlogik kennt weder Ethernet-Details noch eine technologiespezifische Connectivity-Implementierung.

Praxisablauf in fünf Schritten

Kleine Zustandsfolge, vorbereitete Shell-Schritte und klar beobachtbare Ergebnisse

01

Registrieren

Callback früh hinzufügen

02

Synchronisieren

Initialstatus erneut senden

03

Stimulieren

Interface oder Adresse ändern

04

Beobachten

Events, Zustand und Zähler prüfen

05

Recovery

Konnektivität wiederherstellen

Projektquellcode liefert das Gerüst – die Folien konzentrieren sich auf Ablauf und Prüfkriterien.

Ereignisfolge: Interface → Arbeit

INTERFACE → L4-EVENT → WORK

Zustandsänderung wird beobachtet, bewertet und außerhalb des Callbacks verarbeitet

t0 · Start	Callback registrieren und Ereignismaske begrenzen
t1 · Sync	conn_mgr_mon_resend_status() liefert den aktuellen Initialzustand
t2 · Event	Connected oder Disconnected aktualisiert nur den internen Zustand
t3 · Arbeitsauftrag	Thread oder Workqueue übernimmt Transport- und Diagnosearbeit
t4 · Wirkung	Transport folgt dem Zustand; Sensor-Pipeline bleibt aktiv

Mehrfachmeldungen führen dank idempotenter Verarbeitung nicht zu doppelter Wirkung.

Callback klein halten, Arbeit auslagern

MINIMALER CALLBACK

```
event filtern
Connectivity State setzen
Zähler erhöhen
Work Item einplanen

kein blockierendes I/O
keine Transportlogik
keine Sensorarbeit
```

01 Callback-Kontext

nur kurze, threadsichere Zustandsänderung und Übergabe des Arbeitsauftrags.

02 Thread oder Workqueue

Transport aktivieren, pausieren, protokollieren und Recovery koordinieren.

Die Übergabe schützt den Eventpfad vor langen oder blockierenden Operationen.

Drei Anwendungszustände

OFFLINE

Sensorik bleibt aktiv

Proben weiter erfassen

Queue weiter bedienen

Transport pausiert

Disconnect zählen

ONLINE

Transport ist freigegeben

L4 verfügbar

Ausgabe aktivieren

neue Proben senden

Connected zählen

RECOVERY

Zustand stabilisieren

Ressourcen bereinigen

Status neu bewerten

Transport neu freigeben

Proben erhalten

Der Sensorpfad läuft in jedem Zustand; nur der Transport folgt der Konnektivität.

Erwartete Zustände und Zähler

BEOBACHTBARKEIT

Vor und nach jeder Zustandsänderung dieselben vier Sichten vergleichen

Ereignisse	Connected- und Disconnected-Meldungen getrennt zählen
Zustandswechsel	nur echte Änderungen erhöhen den Wechselzähler
Fehler	unerwartete Reihenfolge und Recovery-Fehler separat erfassen
Sensorproben	Erfassung und Queue-Fortschritt bleiben trotz Disconnect sichtbar

Eventzahl und Zustandswechsel müssen nicht identisch sein – Wiederholungen bleiben wirkungslos.

Konnektivität unter native_sim verändern

SHELL-SCHRITT

Zustand gezielt ändern

vorbereitete Interface- oder Adressaktion ausführen

net iface und net cm prüfen

ERWARTETE EVENTS

Folge protokollieren

Disconnected beim Verlust

Connected nach Wiederherstellung

Mehrfachmeldung zulässig

INVARIANTE

Keine gültige Probe verlieren

Sensorerfassung läuft weiter

Queue bleibt konsistent

Transport folgt L4

native_sim ist der Pflichtpfad; Devkits bleiben optionale Varianten derselben Zustandslogik.

Disconnect und Recovery prüfen

Vorher	L4 ist verfügbar, Transport aktiv und Sensorzähler steigt
Disconnect auslösen	vorbereiteten Interface- oder Adresszustand verändern
Offline beobachten	Transport pausiert; Sensorerfassung und Queue bleiben funktionsfähig
Konnektivität herstellen	erwartetes Connected-Event und genau einen wirksamen Zustandswechsel prüfen
Nachher	Transport läuft wieder; Fehler- und Ereigniszähler sind nachvollziehbar

Recovery ist bestanden, wenn keine gültige Sensorprobe durch den Konnektivitätswechsel verloren geht.

Praxisabnahme mit sichtbarem Ergebnis

ERWARTET

```
callback ........ PASS
initial-state ... PASS
disconnected .... PASS
pipeline-active . PASS
recovery ........ PASS
```

01 Zustandsfolge stimmt

Events, interner Zustand und Zähler ergeben eine nachvollziehbare Reihenfolge.

02 Pipeline bleibt neutral

Sensorerfassung funktioniert online, offline und während der Recovery unverändert.

Lernziel erreicht: Konnektivitätsgewinn und -verlust steuern nur den Transport, nicht die Fachlogik.